

installations.

A few examples:

- Design features to avoid downtime since it's not acceptable for SaaS nor self-managed users.
- It is ok to release functionality to SaaS first, as long as the solution is applicable for self-managed too.
- While a feature can be enabled first on SaaS via a feature flag or setting, the underlying implementation must also exist, although disabled, on self-managed.

In line with our SaaS-first principle, some features may be released on SaaS to gain operational experience and apply learnings prior to recommending and supporting customers using it. Features would be present on self-managed codebase but are disabled until General Availability.

For functionality where implementation may be especially challenging without cloud services, for example with AI, self-managed functionality may be dependent on an underlying SaaS service. This allows us to provide end users the same capabilities regardless of deployment type, and not overly constrain our feature set or impose significant operational complexities on each deployment. Product managers need to be aware that this may impact the adoption of these features, as not all customers may be willing or able to leverage underlying SaaS services, such as air-gapped deployments.

Principled adherence to the established knowledge architecture

Exception to this product principle requires CEO approval. Work with VP, Product Management to add your request to the Product Scale agenda describing the situation and request for exception to gain CEO approval.

Our simplicity and SaaS/Self-Managed Parity principles require that we adhere to our established knowledge architecture. Our established architecture is Organization, Group and Project.

- When we need to add capabilities that an admin needs to apply across the entire organization, then we provide them at the organization level.
- When we need to add capabilities that need to apply across all projects in a group, but not applicable to all groups in an organization, then we provide them at a group level.
- When we need to add capabilities that need to apply across a specific project, but not all projects in a group, then we provide them at a project level.
- For capabilities our users want applied to a set of groups, it is tempting to create another aggregation concept for a "set of groups". We do not consider that until it is available both at the project and group level for a few months. The solution is to implement it for all groups at the organization level or for each group in the set individually.
- For capabilities our users want applied to a set of projects, it is tempting to create another aggregation concept for a "set of projects". We do not consider that until it is available both at the project and group level for a few months. The solution is to implement it for all projects at the group level or for each project in the set individually.

Note: This implies that we will take pains to avoid instance-level features as we expect to move all capabilities from instance to organization over time.

Take pains to avoid instance-level features

After making a tier decision on a new feature, we should strive to maximize the number of users who can use it.

As part of this objective, we should avoid building instance-level features when possible. Building at the instance level (in the admin area) leads to a separation between GitLab.com and self-managed and limits your audience to self-managed customers only:

> Historically (and even net-new proposed features) we've often started with an "instance-wide" mindset which then means we need to iterate and adjust features to work at a group-level. This often delays functionality for our SaaS customers and makes GitLab.COM feel like a second-class citizen.

There are factors that may justify an instance-level feature, like engineering efficiency and high infrastructure cost, but we should always have a clear view on how we might bring the feature to GitLab.com and clearly document why we started with the instance-level in the issue.

Deciding whether to add configuration

For a GitLab instance in `gitlab.yml`

Product Managers at GitLab are frequently confronted with the choice of whether to add new configurations or not. These can frequently be times where an outside perspective is important. That's why we've created the option to request a New Config Review.

Here's an example of how to consider whether to add new configuration. Let's say you are proposing we add a checkbox or two radio boxes in a feature dialog box. Think carefully about what users really want. Most of the time, you'll find you really only need one solution, so remove the other option. When two possible choices really are necessary, the best or most common one should be the default, and the other one should be available. If the non-default choices are significantly less common, then consider taking them out of the main workflow for making decisions, by putting them behind an Advanced configuration tab, for example.

Avoiding configurations is not always possible. When we have no choice, the secondary priority is to configure something in the GitLab interface.

A configuration should only appear in a file (`gitlab.rb` or `gitlab.yml`) as a last resort.

- `gitlab.yml`

is the configuration file used by the Rails application. This is where the domain is configured. Other configurations should be moved to the UI as much as possible and no new configurations should be added here.

- `gitlab.rb`

is the configuration file for Omnibus-GitLab. It acts not only as an abstraction of the configuration of `gitlab.yml` for GitLab-Rails, but also

as the source for all configurations for services included and managed within the Omnibus-GitLab. Newly introduced services probably need to be configured here.

When you have to add a new configuration, make sure that the features and services are on by default. Only add a configuration line to either of these configuration files if the feature or service cannot be fully disabled from the admin UI.

For a GitLab CI configuration in `.gitlab-ci.yml`

If the decision to add a configuration follows the principles above, add it to the repository-specific CI configuration options and be sure to default it to the option that results in the best user experience. We are much more liberal with additions to CI configurations than Instance configurations.

Every feature is owned by a group

Features should be owned by one group, including the respective DRIs of that group. Make sure that the documentation metadata and the `features.yml` for your team are kept up to date to make it easier for other teams to find the correct owner.

This principle is important because unowned product features are unsupervised, and continue to accrue technical debt over time. This increases the risk of performance and maintenance issues, which tend to only get resolved once the situation has become critical. In addition, by having clear DRIs for our entire surface area, teams are able to advocate for investment and/or removal of features. If you encounter a feature that does not seem to be owned or documented, work with the team that originally introduced the functionality to decide on ownership. If the feature is large and needs to be broken down, document which elements are owned by which team. If you cannot decide who should own the feature, escalate the decision to the lowest common reporting line between the involved teams. If there are features that no group wants to own, or features that a group no longer wants to own, that feature should be considered for deprecation and removal.

User Experience

A highly usable interface with cohesive workflows and comprehensive documentation is a must to stay ahead of our best-in-class competitors. Work closely with the individuals in UX to achieve our user experience goals. The UX team has a high level of expertise in Product Design, Technical Writing, and UX Research. They can help decipher or decide how to simplify or avoid complexity. While our Product Designers review user interface changes in merge requests, they are not limited to just the UI. Anything that impacts the user journey is relevant to them.

Keep in mind these general user experience principles.

- Strive for simplicity: Using GitLab should be easy. Users should think about the applications they are building and the teams they are collaborating with, not how to make our app work. Here's a great read on "Don't make users think!"
- Depth over breadth: A world-class experience requires deep, powerful, useful features. To remain balanced, we must also identify capabilities that can be deprecated so we are adding depth while encouraging subtractive thinking.

- Better than before: Our MVC principle pushes back against the notion that something must be better than nothing. Instead, we assess if the user experience is better than before by considering its value. Work with a Product Designer to evaluate tradeoffs and minimize deferred UX.
- Timeless design: The user experience should be relevant both today and years from now, so each release should encapsulate the best experience possible. Ask yourself, "How would we build something if we knew this was the last time a team could touch it?"

Additionally, you can familiarize yourself with the UX team mission and GitLab's Pajamas design system principles.

Be Ambitious

Many crazy, over-ambitious ideas sound like they are impossible just because no one else is doing them.

Since we have amazing engineers and a culture of shipping minimal valuable changes, we are able to accomplish many more "impossible" things than other organizations.

That's why we're shipping merge conflict resolution, why we shipped built-in CI before anyone else, why we built a better static pages solution, and why we're able to compete.

How this impacts Planning

Here at GitLab, we are an ambitious company and this means we aim for big things with every release. The reality of taking chances and planning aspirationally means that we won't always be able to deliver everything that we wanted to try in every release, and similar to our OKRs, we believe this is a good thing. We don't want to shy away from challenging ourselves and always want to keep a sense of urgency, and aiming for more helps us do that. Also see the importance of velocity

We arrived at our preference for ambitious planning after measuring our velocity and finding that our velocity was unchanged whether we scheduled ambitiously or scheduled for providing slack.

Discoverability without being annoying

Discovering new features can enhance the experience and unlock significant value for users. And the more users see and try our features, the faster we can get feedback to improve them.

However, excessive feature discovery efforts can become irritating for users. This erodes trust and reduces engagement with other UI elements in the future. Even worse, they might leave GitLab due to this deteriorating experience. Context plays a significant role in how users engage with new functionality. By presenting features in a way that resonates with a user's current situation and needs you increase the likelihood of them using this new functionality.

Work with your product designer to improve the discoverability of your features. The Pajamas Design System has best practices and examples to support feature discoverability. We can also design new patterns. The Growth team can also help you with this, as they think about things like onboarding new users and promoting feature use within the app while supporting, not annoying, the user.

Product Qualified Leads (PQLs)

As the GitLab userbase and team members who work on GitLab continue to grow we need to support both our users and team members by helping to connect users who may be interested in speaking with a member of the sales team to that particular person. We can call this a Product Qualified Lead or a PQL.

A PQL can be further broken down into two types, usage and hand-raise

- Usage: A usage-based PQL is a user or a team (group or instance) that adopted the product to the level that we had data to support that they are statistically more likely to upgrade to a subscription compared to our remaining userbase. When this level of product adoption is achieved by the user or team an alert is triggered for the sales team to follow up with the user and/or team. The level of usage that triggers a usage-based PQL is something that will be determined and agreed upon between Product, Marketing, and Sales as the goal is to generate quality leads for the sales team. When the usage-based definition is agreed upon it will be added here.
- Hand-Raise: A hand-raise PQL is a user who requests to speak to sales from within the product. Our goal is to introduce these hand-raise moments throughout the product at feature discovery moments, or moments where a user is learning more about a paid feature or tier of GitLab that they may find valuable. These moments should be contextually relevant to their usage and non-obtrusive see Discoverability Without Being Annoying. A hand-raise moment in the product should be accompanied by either a trial CTA or a touchless upgrade CTA or both. We want to always provide options to our users as we want them to decide the path that best suits their needs.

Clarifying out what a PQL is not

- A PQL is not a user that has only signed up for the product, they have not achieved a qualified status.
- A PQL is not a trial, a trial is a separate user adoption path. It's important to note that a user could start a trial and then become a PQL or vice versa.

The future vision of PQLs within the GitLab product

Our goal is to develop a world-class PQL system whereby we monitor product usage to understand and constantly iterate on what constitutes a usage-based PQL and provide a unified intelligent interface in the product where users can submit a hand-raise, start a trial or upgrade touchlessly.

By monitoring product usage, usage PQL volume, SAO rate, and ASP we will be able to work in partnership with marketing and sales to ensure we're sending high quality leads to the sales team.

In the product experience, we will develop an intelligent module for feature discovery moments whereby we help recommend what we believe should be the preferred option for the user whether it's a hand-raise, trial or touchless upgrade by updating the default CTA based on their usage of the product along with demographic and firmographic data. This experience will be present on both SaaS and self-managed instances for air-gapped instances the CTAs will provide the user with external URLs to visit to complete the associated step. This experience should be able to be deployed by any stage to further their paid adoption rate.

Drive Product Usage

Users can only experience GitLab's value when they actively use the product features. Therefore the Product team's mission isn't only shipping features and building products, but also driving usage and delivering value.

There are two frameworks we use to think about driving GitLab's product usage: we use the AARRR framework to think about how to drive a single feature's usage, and use the Customer Adoption Journey to think about cross-adoption of product features. These two frameworks are also interconnected with each other.

Single Feature usage: AARRR framework

AARRR stands for Acquisition, Activation, Retention, Revenue, and Referral which is often referred to as "Pirate Metrics". These five words represent the customer journey and the various means a product manager may apply Product Performance Indicators to drive a desired behavior in the funnel.

While the AARRR framework is commonly used to drive overall active users, it is also a great way for PMs to think about how to drive feature usage.

- Acquisition measures user actions that show awareness of the feature
- Activation illustrates a user has begun to apply the feature
- Retention is the continued use of the feature over time
- Revenue captures the monetary value acquired from feature usage
- Referral focuses on measuring behaviors that drive users to encourage others to consume the feature

```
mermaid
classDiagram
    Acquisition --|> Activation
    Acquisition : Are users aware of the product or feature set?
    Acquisition: Measurement (Insert Metric)
    Activation --|> Retention
    Activation : Are users applying the feature?
    Activation: Measurement (Insert Metric)
    Retention --|> Revenue
    Retention : Are users applying the feature over time?
    Retention: Measurement (Insert Metric)
    Revenue --|> Referral
    Revenue : Are users paying for the features?
    Revenue: Measurement (Insert Metric)
```

Referral --|> Acquisition

Referral : Are users encouraging others to use the feature?

Referral: Measurement (Insert Metric)

Add AARRR funnels for your stage or group's Product Performance Indicators directly with mermaid markdown. It's easy if you use this live editor.

Product managers can use these various states to prioritize features that drive a desired action. This could mean focusing on the Activation metric to drive awareness and generate more top of funnel leads. As an example, in the Release stage the Release Management group tracks actions on the Release Page in GitLab. Users that view a Release Page have been acquired and those that create a release on a Release Page are activated users. The Product Manager can choose to target features that drive users to view the Release Page more, resulting in a greater interest in the number of users that become activated and create their own Releases.

Multi-feature usage: Adoption Journey

GitLab is a complete DevOps platform. Our customers get the most value out of the GitLab product when they use multiple features together. Below is the most common path our customers follow to adopt GitLab's product stages.

As PMs, in addition to driving usage of individual features, we should also proactively think about how to design product and user experiences to help users adopt more stages and features, therefore benefiting more from using GitLab.

```
mermaid
graph TD
    id1(Create - 99%) --> id2(Plan - 91%)
    id3(Verify - 84%) --> id4(Secure - 36%)
    id1 --> id5(Manage - 35%)
    id1 ==> |Golden Journey| id3(Verify - 86%)
    id3 ==> |Golden Journey| id6(Release - 54%)
    id3 --> id7(Package - 7%)
    id6 --> id9(Monitor - 8%)
    id6 --> id10(Configure - 2%)
    id6 --> id11(Software Supply Chain Security - 28%)
```

- The percentage here is defined as % of monthly active paid ultimate-tier self-managed instances that adopted that stage. The data is captured directly in the Golden Journey Paths chart (which has been deprecated).

- The Golden Journey: The bolded path is the "Golden Journey" where we observe as the most common stages adopted by paid customers and serve as foundation to adopt other stages: it starts from Create, then goes to Verify and Release. If the Golden Journey is complete, all stages of GitLab are available for use. Our biggest opportunity is to improve Verify to Release adoption rate.

Note: There are numerous potential variants to this adoption journey, but it's important to keep this representation simple and consistent. Please check with David DeSanto first before

making any changes to the adoption journey image.

Flow One

Shipping only MVCs can result in a large set of loosely connected pieces that don't necessarily combine into a single, great user experience.

An obvious solution to this would be to plan out the future in detail, creating a long-term detailed plan. However, this is unwanted as it can restrict your flexibility and ability to respond to changing needs or feedback.

Flow One offers an alternative. You draw out a workflow consisting of MVCs (that can be shipped individually). The workflow should only cover a specific, narrow use-case, and nothing more.

This means you:

- avoid creating an inflexible, long-term plan
- can more easily construct a full feature/ capability, which is more easily marketed
- can provide context to each individual change ("we need this as part of X")
- can continue to ship MVCs
- work concurrently on several items, none of which are blocking

Flow One should cover the first iteration of a particular workflow. After this, individual MVCs can be introduced to expand the use-cases or loosen the assumptions (e.g. from a feature that can only be used if you're using feature branches, to one that works for other git strategies).

Data-driven work

Using data to learn from our users is important. Our users are spread across GitLab.com and self-managed instances, so we have to focus our efforts on learning and providing benefit to both when we decide to collect more data, or build and use additional analytics tools. If we do this, we can help make the rest of the company successful as well. This means that we should:

- Build and use tools that work for both GitLab.com and self-managed.
- Start from a question, and build / collect what we need to answer that question. This avoids wasting time with data we don't need.
- Use and improve existing tools we have inside of GitLab before leaning towards off-the-shelf products.
- Our customers, sales team and customer success teams all benefit greatly from similar insights into their usage as the product team does. Make things that help all of these people.

No artificial limits in Core

Per GitLab Stewardship, we will not introduce artificial limits in Core. Artificial means arbitrarily setting a small number (such as: 1) as a limit on a given GitLab object category, that would incur no additional effort or cost had we chosen a larger number. The additional effort includes product, design, and engineering effort to create the feature in the first

place,
and to maintain it over time.

For example, GitLab Core has the issue board feature in every project.

In GitLab EE, each project supports multiple boards.

This does not mean that Core has an artificial limit of one board per project, because there is additional effort to manage multiple boards such as supporting the navigation interface, and all the associated engineering work.

This principle does not apply to our SaaS offering as limits are occasionally introduced to limit our hosting costs and protect other users from potential abuse. As an example we have shared runner minute quotas and implement rate limiting.

Avoid enforced workflows but allow enterprise flexibility

We're discussing enforced workflows in this issue.

Enforced workflows should be avoided in GitLab. For example, there are three issue states (Open, In Progress (as of 10.2), and Closed), and any issue should be allowed to transition from one state to any other state without workflow restrictions. (Roles and permissions is a separate concern.)

- Enforced workflows restrict GitLab to a smaller number of use cases, thus reducing the value of GitLab.
- Enforced workflows require overhead to maintain in the product. Each new feature must account for any existing enforced workflows.
- We should trust our users to use GitLab responsibly, giving them freedom, instead of imposing enforced workflows that we think made sense at the time of design and implementation.

A comment on Hacker News perfectly details what can go wrong when enforcing workflows:

"The down side for the true end-users, those who actually use the software day-to-day, is that most business processes are awful. If your experience is the hellish existence that I see strolled about on threads where JIRA comes up ...:

1. Your admin(s) set it up once and hasn't bothered to iterate on those workflows.
1. The business mapped their autonomy stripping processes onto JIRA intentionally. I'd guess that most of your work experience is similar. Process stifled nonsense."

But that comment also specifies the advantage:

"JIRA's most powerful feature is that it affords for mapping businesses processes onto software.

This is incredibly compelling to enterprise customers. Software that enforces workflows, procedures

and requirements can be an incredible lever and JIRA's price point makes build vs buy decisions an absolute no-brainer."

We should ensure that GitLab makes it easy to help with enterprise workflows:

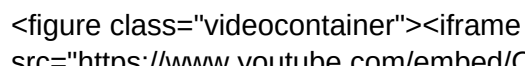
- When starting a branch with an issue (number) we link it to the branch.
- When merging an MR you automatically close the issue(s) it fixes.
- In GitLab CI you can define your deployment stage progression (staging, pre-production, production) including manual approval.
- We run quality and security tools automatically and provide dashboards to check status instead of making it a step in the process.
- We limit the impact of mistakes with incremental rollout and automatic rollback.

When considering a customer need for enforcement or limitations:

- Deeply understand and document the underlying customer problem. It's our responsibility to understand the need we are solving for before we even consider imposing controls.
- Solve for individual use cases first. Trying to solve a specific problem with a non-specific solution is risky and not iterative. Instead, start with a single use case and build a specific, non-enforced solution in GitLab.
- Consider the smallest group of users first. Don't reach for instance-wide controls, but instead iterate from the smallest segment possible (e.g. a sub-set of projects).
- Provide simple workarounds and overrides. Consider extreme scenarios like recovering from a SEV-1 incident; there should always be a simple, fast escape hatch.

As an example, customers requested instance-wide enforcement through required CI jobs. Doing this would have been a mistake. Instead:

- We understood their problem more deeply, and realized we could build capabilities to perform these checks in existing primitives (like external rules in MR approvals).
- We limited the scope of the problem, and avoided any limitations at the instance-level. Instead, we planned to keep the impact as small as possible by requiring customers to scope this feature to only projects with a particular compliance framework.
- We intentionally planned for workarounds. A developer should be able to override these limitations inside merge requests, such as with two-person approvals. They should also be able to create subgroups that aren't subject to these controls.

While workflow enforcement should be avoided in most cases, there are organizations that rely on enforced workflows for a variety of reasons. These organizations have problems adapting existing workflows as they move to GitLab and as a result, we should consider allowing some enforcement at the group level in order to balance team efficiency and organizational policy. Page 79 of Accelerate outlines "teams that reported no approval process or used peer review achieved higher software delivery performance." As we implement features to allow organizations to enforce workflows, we should do so at the group level and default them off. GitLab should be a product teams use to accelerate product development, but flexible enough to solve for requirements of all sizes of organizations.

Prefer small primitives

Small primitives are building blocks in GitLab. They are an abstraction not at the technical level, but truly at the product level. Small primitives can be combined,

built-upon further, and otherwise leveraged to create new functionality in GitLab. For example, the label lists in issue boards use the smaller primitive of labels.

They are especially powerful because they usually take less effort and provide higher leverage than you would get from a more "complete" but standalone feature. Think of how simple Unix command line utilities can be chained together to do really complicated things, much easier (and certainly more flexibly) than you could have done with a dedicated tool.

When iterating on GitLab, strongly consider using small primitives instead of creating new abstractions, especially when considering MVC features that will provide the foundations for further improvements. To do this you can start with easy to apply concepts that meet the needs of intermediate to advanced users; from here document the usage clearly and be sure to think about discoverability. The UX can very often be refactored or enhanced later when there's a demonstrated need for refinement, onboarding for less sophisticated users, or other new abstractions needed that were identified through real-world usage.

Component Principles

In GitLab the product, it is sometimes the case that optional software or infrastructure is required to enable new capabilities. Some examples include:

- Infrastructure in the form of Runners are required to enable use of GitLab CI/CD
- Infrastructure and software in the form of Elasticsearch is required to enable Advanced Search
- Software in the form of the Kubernetes Agent is required to enable GitOps pull-based workflows

The following are best practices we consider when building such components.

Start with enabling Developers

As we learned with GitLab CI/CD, the ability for developers to quickly attach needed Runners to enable their own use of GitLab CI/CD allowed for rapid adoption of GitLab CI/CD within an organization. When considering the workflow to enable additional capabilities, start with enabling developers first. A guiding principle should be low-friction developer enablement, which will positively impact adoption.

Build for production use, not demo use

As we learned from the certificate-based Kubernetes Integration, building starting experiences that support demoing a getting-started process doesn't necessarily translate into real usage. For example, the certificate-based integration lacked strong security primitives and the ability to manage the integration on an ongoing basis. As a result, we should build capabilities that target real-world production use first, even as part of MVCs.

Integrate other applications thoughtfully

GitLab's vision is to be the best single application for every part of the DevOps toolchain. However, some customers use tools other than our included features, and we respect those decisions. With this in mind, it's sometimes valuable to integrate with 3rd-party services and products to help bridge the gaps in their toolchain. While a single application is the best approach, multiple applications that work well together is better than ones that don't.

With this in mind, below are some product guidelines to consider:

- Don't clone everything
 - Don't feel like we need to clone every feature of an integrated product. Ultimately, this approach costs us the most possible resources, while creating an experience that will always be sub-par compared to the native one offered by the other application. We should focus our efforts on places where we can deliver value by helping bridge the gap between the two applications.
 - Here are some potential (completely theoretical) examples, highlighting the difference between cloning vs. bridging:
 - In an event log, surface only recent activity and link to a full log on the other system, instead of trying to display all records inside of GitLab.
 - In an issue/ticket, offer only the most used functionality, like commenting or status changes, instead of trying to replicate all possible actions.
- Prevent broken workflows for free
 - Based on the quote above: "While a single application is the best approach, multiple applications that work well together is better than ones that don't."
 - Features and UX that might break a customer's workflow should be offered freely. In particular, we should offer features that help customers avoid productivity antipatterns like allowing multiple sources of truth (multiple wikis) or duplicating work (not showing work activity on an issue).
- Charge for adding value to an interaction
 - By having such an incredible breadth, GitLab has the opportunity to add value to external features that they could never otherwise have, if not integrated with GitLab. For example, our excellent To Dos feature puts actionable work at the fingertips of developers, and it's a feature somewhat unique to GitLab. Many other tools make it harder to identify work that needs immediate action, so by integrating those tools with this part of GitLab, we can make them even more powerful than they'd be alone.
- Consider the right buyer for pricing
 - While we typically think about who is using the functionality as an indicator of our buyer, this may not be appropriate for your integration.
 - When an integration is driven by an organizational requirement (such as having all teams in the company use Jira), Buyer Based Tiering likely indicates that the integration is for multiple team (Premium) or strategic organizational (Ultimate) use.
 - When an integration is extending GitLab capabilities, the buyer is more likely to be the end-user, similar to how we'd think about pricing other GitLab functionality.
- Low level of shame still applies here
 - When we start to integrate with another product, it's almost like opening a whole new Category of work. With this in mind, a friendly reminder that low level of shame applies, just like it would anywhere else. The integration may not be very valuable today, but it's just a starting place, and we should use our first iteration to validate what belongs in the next one.
- Track the maturity of the integration
 - Like any other category of work, we should track the maturity of the integration somewhere that's visible. Consider adding your integration to the Ecosystem Integrations direction page,

which contains a table that tracks our high-priority integrations and their maturity levels.

- Respect security and permissions

- External applications may have security and permissions concerns that we don't have in our application. We should take careful consideration of these, since ignoring them could be potentially disastrous for our users. For example, if we ask a user for an administrative token with full access and then display content in our UI based on those privileges, we could potentially display sensitive data that shouldn't leave that application.

- Navigation

- Integrations should be as discoverable as any native GitLab feature, if they're enabled. In cases where the two features are mutually exclusive, we should allow the user to hide our native functionality to prevent confusion. For example, having two project management tools is a (generally) bad idea. Collaborators may end up filing issues in the wrong tool, slowing down their velocity and worsening their overall experience. Reducing velocity is antithetical to the value GitLab provides.

- When features aren't mutually exclusive (there's nothing wrong with using multiple security scanning services, for example), we should group navigation to those integrations near our own.

- Use empty states for feature discovery

- Consider adding an empty state that highlights the availability of this integration. If someone is not using the GitLab feature and also not integrating their tool of choice, that's the worst possible experience. We should respect the choices of our users, and if they've decided to use a 3rd-party tool that we integrate with, we shouldn't hide that option in a dark corner.

- This is also a great opportunity to educate the user on what GitLab can offer relative to other products. It's the best of both worlds: promoting our own features while helping them improve their current experience, regardless of choice.

- Use our own design language

- Just because we're leveraging functionality from a 3rd-party service doesn't mean we should allow that service's UI to dictate what ours looks like. GitLab has its own design language, and by keeping it cohesive across the application, we create a significantly better experience for our users.

- Be transparent about the data source

- When data is populated from an external source, make it clear that it comes from elsewhere. This prevents confusion, reduces cognitive load, and ensures that users are well informed about the choices they're making when interacting with the application.

Avoid plugins and commercial marketplaces

Note - We intend to provide a place where everyone can contribute, such as code snippets, project templates and CI components in the CI/CD Catalog. Marketplaces are traditionally transaction based, where users are purchasing solutions. GitLab CI/CD components on the other hand, will offer an in product experience for consuming components for YAML configuration from a library of GitLab maintained components.

Closed source software vendors commonly depend on plugins and commercial marketplaces because:

1. These allow third parties the ability to add functionality without needing access to the underlying codebase.

1. Third parties can charge for the plugins, which gives them an incentive to build them.

Because GitLab is an open core product, third parties can add functionality directly to GitLab.

Adding directly to the GitLab codebase (as opposed to building a plugin) may mean more work for them and will limit the ways in which they can charge for that functionality.

However, for users of GitLab, this has significant advantages:

1. They are more likely to use this functionality, because it's distributed to all users by default.
1. The functionality always works out of the box, requiring no additional software.
1. Users don't have to search for what plugins are available, or test multiple options to see which one works the best.
1. Installations can't have varying combinations of plugins, making it easier to use and support the product.
1. Administrators never have to avoid an upgrade for fear that it might break an important plugin since all code is tested together.

And for developers of GitLab including the third parties, this has significant advantages as well:

1. Their work isn't limited in functionality, because they're changing the code of the product itself.
1. Their code will be tested along with the rest of the product as things change refactoring is easier, reducing the likelihood that it suffers from software atrophy because something is hard to change.
1. Community effort is not wasted on developing multiple plugins that all ultimately do the same thing and compete with each other for usage.
1. Developers don't have to sacrifice velocity in favor of predictability, since they don't have to worry about breaking the plugin API.

Overall, we believe that this approach creates the best possible experience for both the users of and the contributors to GitLab, and to that end we encourage people to contribute functionality to GitLab directly.

If adding code directly to GitLab isn't an option, we encourage third-parties to integrate through our APIs.

Note: GitLab does support plugins that respond to system hooks, which tie directly to application events and are primarily used for administrative purposes such as auditing, logging, and other administrative tasks.

Product and feature naming guidelines

Give products and features descriptive, not distinctive, names

GitLab is a DevOps Platform, not a collection of DevOps point solutions; the naming of GitLab products and features should reflect this. Adopting descriptive names brings other advantages:

- It's efficient. Descriptive names are generally not eligible for trademark registration, avoiding the time, effort, and expense of clearing, filing, and maintaining trademark registrations.
- It's inclusive. Descriptive names are most accessible to a global audience because they can

be directly translated into other languages, reducing the risk of loss of meaning inherent in translating abstract, metaphorical, or colloquial names.

As an example, CI/CD Components Library is a descriptive name while CI/CD ATOM is a distinctive name.

Exceptions to this principle are considered in limited circumstances - if a product or feature is a differentiator in the market, adopting a descriptive name risks it getting lost in the sea of same. Here, a distinctive name may be justified. To discuss an exception, reach out to marketing in Slack.

Use prepositions when referring to third-party products and services in names

When naming a GitLab extension, plugin, app, or integration for a third-party product or service (a "tool"), either include the third-party service name with a preposition like for, or don't include it at all. A preposition is important as it indicates that the third party isn't officially affiliated with the tool. For example, our integration with Jira Cloud is called GitLab.com for Jira Cloud.

Provided we use the tool's full name in marketing materials and technical documentation, as exceptions to this general principle we can: (1) refer to the tool solely by the third-party service name in places where we list GitLab's tools in product; and (2) refer to the tool solely as "GitLab" in the third-party product's app directory. For example, we refer to the GitLab for Slack app as "Slack" in our lists of GitLab integrations, and as "GitLab" in the Slack App Directory.

For more guidance on the feature naming process, see naming features.

Next Generation

While our big, hairy, audacious goal spans all development processes, personas, and use-cases, there are primary targets in each one of these venues.

When considering prioritization we should first aim to provide complete maturity for developers building cloud native applications in a modern way prior to moving to other development methodologies, personas, and application types.

Modern first

When developing features to compete with existing competitors, make sure to solve problems for modern development teams first, and then see what's missing for legacy teams. e.g. For project management, make great project management capabilities for teams doing conversational development, lean, or even agile development before doing Scaled Agile Framework (SAFe) or waterfall.

It's important that modern first does not mean non-modern never. It means that we should first learn how teams are using the feature in a modern way, and then see what's missing. The modern way provides the path forward, and then we can add customizability or the path to modern for teams who are not quite there yet.

Developer first

Our strategy includes going after a lot of new personas, going from developers to operations, security, product managers, designers, etc. But when developing features in these new areas, it's important to remember to start with the developer. If we can make security great for developers and then great for security professionals, we'll be much more successful.

Cloud-native first

Development teams deploy to tons of different platforms, from bare metal to cloud VMs to cloud-native Kubernetes clusters. We build features for cloud-native first, and then support the rest. This allows us to focus on where development is going, and deliver solutions that every company aspires to use eventually, even if they're not ready to today.

Prioritize current adopters

By focusing on next-generation development flows, personas, and use cases - we build features and experiences where our initial users are in the relatively small population of early adopters. While we might build experiences to support them today, we presume there will always be a much larger population of future users of these experiences. Therefore, we optimize GitLab to support the larger number of current and future adopters of next-generation principles - those who are beginning to operate in the workflow (modern), team setup (developer first), or application architectures (cloud native) we support. We focus our investment in the most modern workflows that will best support those current adopters. This will come at the cost of sustained investment in initial workflows for early adopters. When doing so we will ensure we clearly communicate with our users what the preferred path is.

For example - We first provided the ability to attach Kubernetes Clusters via the Certificate method. After realizing this wasn't optimal for production use cases we added the GitLab Agent for Kubernetes method of cluster attachment. As soon as we were certain that the certificate method was no longer the preferred method we communicated via our docs and in the product that the Agent was the preferred path for current adopters. This should not mean an immediate deprecation, but a clear signal that the legacy method will become deprecated once the alternative approach is able to substitute it.

Customer Support on all features {support-all-features}

We provide customer support to paying customers on all features that are in the tier of their paid license as well as any lower tiers.

This means a feature in Core, should get customer support in all paid tiers.

For example, when there is a feature that is available in the lowest paid tier, but only has customer support in higher tiers,

the feature should be removed from that lowest paid tier.

In accordance with our stewardship policy we can never remove features from Core into paid tiers,

but we can build additional features around it which are available to paying customers only.

SECTION: product/ai/_index.md

This page serves as a resource for team members looking for information about working on AI features at GitLab. The first iteration of this page is a list of links, loosely organized. This will improve as we build up our best practices and consolidate/improve our documentation.

Getting started on an AI feature

Suggest an AI feature: Issue template for suggesting or requesting an AI feature

Design and UX research for AI features

Pajamas AI guidelines: How to design the user experience for AI features. Includes design patterns and guidelines.

UX research in AI space: Helps team members evaluate the usefulness and usability of AI features.

UX maturity guidelines: Suggested UX validation to move up in AI feature maturity.

Evaluating Quality of AI features

The AI Model Validation group has guidance (internal link) for testing your AI-powered feature, which includes model validation via the Central Evaluation Framework(CEF) (internal link). You can make a request to have your feature evaluated.

The UX Research team maintains a usability evaluation and scoring process to ensure ease of use.

For fast, qualitative feedback on AI response quality, as well as to get an idea of what kinds of prompts users write, you can run a UX Chat Bash.

Special considerations for AI features

AI ethics principles: Principles to guide our decision making as we continue to build AI features into GitLab and to ensure that these features properly embody our values.

AI data usage and privacy: Public facing docs about how we use and protect data.

Engineering AI features

Development process AI features

List of language models: All the models we are using to power features.

Process for choosing or switching AI vendors

Support engineering AI features

Support AI workflow: This page has information for handling tickets related to AI features.

Current AI features and teams

AI features by tier: All features that are available to use, and how to access them.

AI teams

SECTION: product/ai/continuity-plan.md

Purpose

Before selecting a third-party AI vendor, GitLab evaluates whether the vendor uses GitLab or GitLab customer content for developing, training, re-training, or fine tuning the vendor's language models.

The AI Continuity Plan outlines the process GitLab uses to (i) identify when third-party AI vendors modify their terms or practices to be inconsistent with the foregoing commitment; and (ii) transition to and implement alternative AI vendors that meet the foregoing commitment.

Roles & Responsibilities

Role	Responsibility
GitLab Team Members	Responsible for following the requirements in this procedure
AI Powered Stage	Responsible for implementing and executing this procedure
Legal & Corporate Affairs (LACA)	Responsible for approving significant changes and exceptions to this procedure
Security Risk	Responsible for conducting periodic Business Impact Analysis and applying Critical System Tiers to Tech Stack Systems

Procedure

Use the Subprocessor List to Identify Changes to AI Vendors

GitLab documents AI vendors who process customer data on our Subprocessors list, which also allows anyone to sign up for notifications for Subprocessor changes. Those subscribed to this distribution list are notified whenever an event occurs that causes GitLab to amend its Subprocessor list, which may include the removal of AI vendors whose terms are likely to impact customers negatively.

Process to Add AI Models

GitLab continuously evaluates the AI market to identify both proprietary and open source AI models that might be useful to power present and future GitLab AI features.

GitLab has a dedicated model evaluation group that develops technologies to test new models. This group is chartered to identify, select, and validate the use of AI models for quality and performance to ensure they meet GitLab's needs. Read about their process [here](#).

When a model is validated, a recommendation is made to the AI Framework group to implement the selected model within the GitLab AI Architecture. This triggers a variety of tasks including:

1. Procurement of new AI Model

1. This includes Privacy, Security, and contractual reviews via GitLab's existing procurement process.

1. GitLab Legal and Corporate Affairs evaluates any contractual terms relating to customer data usage with the AI vendor who offers the chosen model. They also negotiate terms as necessary to comply with our AI data retention policies and customer data usage policies.

1. If the AI vendor requires an account, a GitLab corporate account will be provisioned and

added to the GitLab Tech Stack. This allows GitLab to track which employees have access and limit access to only employees with a need to know. GitLab separates development and production account access whenever possible to reduce risk.

1. Notification of new AI Model

1. The GitLab Subprocessor distribution list is leveraged to provide 30-day notice of new AI vendors. This waiting period provides customers with their contractual right to object to any new or materially different third-party processing of their data and a chance to terminate the services where third-party processing cannot be disabled. It also provides customers a window to audit planned usage and, where possible, opt-out of using unwanted AI features.

1. Implementation of AI Model

1. The AI Framework group implements support for the new AI model using the provisioned GitLab account and following the GitLab development flow.

1. Coordination occurs with various teams who have implemented or will implement GitLab AI features based on this new AI model. This includes collaboration with the Cloud Connector group to ensure the model can be supported with all distributions of GitLab.

1. Rollout of AI model

1. After the 30-day subprocessor notification period passes and the code changes pass the readiness review, then GitLab will proceed with rollout of the new AI vendor. It's common for GitLab to leverage feature flags and percentage rollouts to control and monitor the enablement of new functionality.

Process to Switch AI Models

GitLab has developed an AI abstraction layer, which GitLab uses to switch the underlying models that power our AI features. This abstraction layer helps reduce the technical complexity of changes, increases the speed at which GitLab can react to changes or failures from AI vendors, and standardizes the way GitLab Rails application interacts with AI models. The AI Abstraction layer also provides centralized telemetry and monitoring to help ensure that all AI vendors meet the production needs of GitLab AI functionality.

When GitLab identifies the need to switch an AI model powering GitLab's AI functionality, assuming the model is from an existing AI vendor and there is no other material change in the processing, no notification of change is triggered to customers. However, any relevant documentation will be updated to reflect the AI model change as part of our Transparency policy with AI functionality.

If the model is from a new AI vendor, the Process to Add AI Vendors (described above) is triggered.

Process when AI Vendor Terms Materially Change

GitLab has established subprocessor policy and term notification and review processes. When an AI vendor changes their terms, GitLab Legal is notified and reviews the changes to determine if they materially impact customers. In the event that term changes materially negatively affect customers, we will proceed with the following:

1. Consider temporarily disabling the affected AI functionality using the impacted AI vendor's models. Should disabling be necessary, GitLab will post a status page update indicating reduced functionality. Concurrently the following steps will be performed:

1. Evaluate any alternative models that are already implemented in the AI Abstraction

layer. Should an alternative model be a candidate for replacement, a small scale test will be performed to ensure the model performs against the relevant use case. This will be performed by both the implementing team of that AI functionality and the AI Model Validation group. This ensures that AI functionality does not switch to a model which could significantly negatively impact the quality or performance of the feature.

1. If the alternative model is validated as a reasonable replacement for the AI functionality, then code changes will immediately be implemented to route traffic to this new model. In most cases this will not require a GitLab version release, as the GitLab AI Architecture can be deployed outside GitLab's monthly release cadence.

1. Should no relevant replacement model currently exist, procurement will begin to identify and select a new AI vendor with a model suitable for GitLab's AI functionality needs. This will then trigger the Process to Add AI Vendors (described above).

Process for AI Vendor Security Incident

As with any security incident reported to GitLab, the Security Incident Response Team (SIRT) will engage to triage and facilitate remediation. Read more about the SIRT incident response process.

Communication Plan and Role Assignments

When it comes to transitioning AI vendors to support GitLab's AI features and functionality, communication is of the essence. A plan is essential because it puts all team members on the same page and clearly outlines all communication.

Documents should all include updated team member contact information and each team member should understand exactly what their role is in the days following the triggering of the AI Continuity Plan.

Each GitLab team should be trained and ready to deploy in the event of a situation triggering the AI Continuity Plan.

Distribute and Verify the Plan / Approval from Senior Management

GitLab documentation related to all procedure and guidelines that detail AI Continuity must be reviewed, updated, and formally approved by Product, Engineering, and LACA leadership.

After developing basic guidelines, this plan will be distributed as a work in progress to the core team.

The core team will review to verify that all technical details are covered and no deficiencies exist.

The core team will review and add guidelines so that all related DRIs are clear on what is required in all situations.

Managers or team leads will share the relevant parts of the plan to all GitLab team members based on their role and department, and verify that they know what to do if the plan is triggered.

SECTION: product/ai/ux-maturity.md

Summary

The following guidelines focus on the UX aspect of the maturity of AI-assisted features. Other aspects, like stability or documentation, should also be taken into account to determine the appropriate feature maturity.

To evaluate the UX maturity of AI-assisted features, use three criteria from the Product Development Flow:

1. Validation: Problem validation: How well do we understand the problem?
1. Validation: Solution validation: How well does the solution address the problem?
1. Build: Improve: How successful is the solution?

Suggested validation for AI features

Criteria/Maturity	Experiment	Beta	GA
Problem validation	Can be based only on assumptions (evidence not required).	Mix of evidence and assumptions.	Only evidence or no high-risk assumptions.
Solution validation	Not required.	Solution validation with 6 internal users.	Solution validation with 6 external users.
Improve	Not applicable (no data before launch).	Quality goals set by the team are reached.	Quality goals set by the team are reached.
AI Output and Usability Quality	Not required.	Quality goals set by the team are reached.	Quality goals set by the team are reached.

Validation: Problem validation

In order to determine a solution, it's essential to understand the customer problem. See guidelines for AI problem validation.

Validation: Solution validation

AI solutions follow the standard solution validation process, with some additional considerations:

It can be beneficial to simulate the AI system's output in testing early to inform engineering efforts.

AI systems can produce inaccurate or unpredictable output. It's important to collect user feedback about AI errors, trust, and risk.

See guidelines for AI solution validation and AI Usability Metric for more information.

Build: Improve

How successful is the solution? To answer this, in this context of feature UX maturity, teams should look beyond feature usage as the success metric and try to include usability signals. High usage doesn't necessarily mean the feature is successful. Usability signals help assess solution success in terms of how useful, efficient, effective, satisfying, and learnable it is.

It's also important to include AI response accuracy in your success metrics. AI-powered features can generate a response or output that is incorrect, irrelevant, or harmful. The risk of an incorrect response depends on the feature. It's important to test the AI system's responses as part of a formative evaluation. For example, you can have one or more expert evaluators (internal or external) test different scenarios to assess the AI responses.

Evaluating the Quality of AI Output

While GitLab does not directly control the models we use from our AI Vendors, the output of these models can be influenced by various techniques we do control (prompt engineering, RAG, context enrichment, etc). End users do not care how this works, so the quality of output is directly perceived as quality of the feature. For that reason we've provided the following guidelines to help teams navigate output quality confidence and risk.

Quality confidence should be a measure of how certain we are that the feature will or won't product harmful or incorrect results that damage the user experience. Risk should be a measure of how likely the feature will produce harmful or incorrect results.

Teams can use UX Research guidance to determine their confidence level. Here are additional suggestions:

High confidence = strong evidence from 2 or more customer facing sources (see below).

Medium confidence = evidence of good performance from at least one customer facing source and little to no evidence of bad performance.

Low confidence = no customer facing evidence of good performance OR evidence of poor performance.

Risk is a measure of how much potential harm can a feature cause to a user, an organization, GitLab, or the world. Creating vulnerable code, risking bad press, generating false information are examples of risk.

We have a variety of tools and processes to help you navigate various states of confidence and risk they include:

UX Bash

Central Evaluation Framework which is supported by the Model Validation team

Intro to Prompt Engineering

Note: For Duo Enterprise, the Definition of Done) is the SSOT.

High Confidence & Low Risk

When an AI-powered feature has high confidence in the output quality and low risk of producing harmful or incorrect results, the feature can rely primarily on the standard feature maturity definitions. In this case an evaluation dataset is not required, however the feature should have basic guardrails like unit tests and can rely mainly on the existing AI Framework and AI Gateway.

Consider running a UX Bash with at least 10 external users validating the quality of the output. We have a framework for how to run them to evaluate output quality.

You should have a baseline understanding of the quality of your feature.

- Above a threshold where users at least find the output helpful

- Output quality should not cause user concern (insecure, offensive, risky, etc)

- Have a proposed quality threshold based on your initial eval dataset.

Low Confidence & Low Risk

When an AI-powered feature has low confidence in the output quality and low risk of producing harmful or incorrect results, the feature can rely primarily on small scale and basic evaluations, usually performed by developers during development and within unit tests. In this case, an evaluation dataset is not required, however plans should be developed to improve confidence. Features in this state should not be moved to GA before increasing confidence.

It's recommended you have a UX Bash with at least 10 external users validating the quality of the output. We have a framework for how to run them to evaluate output quality.

Develop a dataset of at least 100 prompts (user inputs and desired LLM outputs) that proxy expected production use of your feature. Provide examples that focus both on desired and potentially undesirable behavior from the LLM.

- Primary use-cases for your feature should be included in the dataset

- Eval dataset should be representative of realistic customer usage

- We strongly prefer basing evals on historical data from GitLab based data whenever possible.

Unknown Confidence & Unknown Risk

When an AI-powered feature has unknown confidence in the output quality and unknown risk of producing harmful or incorrect results, go back to the root user problem you are trying to solve.

It's helpful to define:

- What the feature should do

- What the feature should NOT do

- Examples of high quality output

When in this unknown territory, you should not proceed with releasing an AI feature. Contact the AI-powered stage (sai-powered) in slack to get ideas of how to start removing uncertainty.

High Confidence & High Risk

When an AI-powered feature has high confidence in the output quality and high risk of producing harmful or incorrect results, the feature must develop an evaluation dataset.

It's required that you have a multiple UX Bashes with at least 10 external users each time validating the quality of the output. We have a framework for how to run them to evaluate output quality.

Develop a dataset of at least 500 evaluations that focus on the areas of risk you've identified.

- The dataset is supported in the CEF and included in the daily CEF run.

All supported use-cases and/or programming languages supported have datasets for evaluation at scale (should be hundreds of evaluations or more)

Ideally, run a final UX bash to ensure real end users agree with the eval quality threshold.

Obtain explicit approval from leadership before considering the feature to go GA.

Low Confidence & High Risk

When an AI-powered feature has low confidence in the output quality and high risk of producing harmful or incorrect results, the feature should not proceed until the team increases confidence of the output quality.

It's required that you have multiple UX Bashes with at least 10 external users each validating the quality of the output. We have a framework for how to run them to evaluate output quality.

Develop a dataset of at least 100 evaluations that focus on the areas of risk you've identified.

The dataset is supported in the CEF and included in the daily CEF run.

Obtain explicit approval from leadership before considering the feature for anything past experimental, off by default release

SECTION: product/ai/ux-support.md

Getting UX Support for AI Related Issues

What UX can help with

The UX team supporting the Data Science section is small and growing. Our aim is to support the following:

- deep understanding of AI capabilities and user needs to ensure problem and human-centered solutions

- UI design for platform level AI functionality such as onboarding, access & settings, and Duo Chat UI

- best design practices and research methods that work well in the AI space

- maintaining consistency across the platform by leading or collaborating on Pajamas AI-human interaction components and guidance

Group Product Designers support AI features in their space, including problem and solution validation and design activities. AI team Product Designers can provide decision support and guidance in this space, primarily from a best practice perspective.

How to get help from the UX team

Use the Product Development Flow for process activities and workflow labels

If you are in the Da